

INTRODUÇÃO À TEORIA DA COMPLEXIDADE

João Pedro Oliveira Batisteli

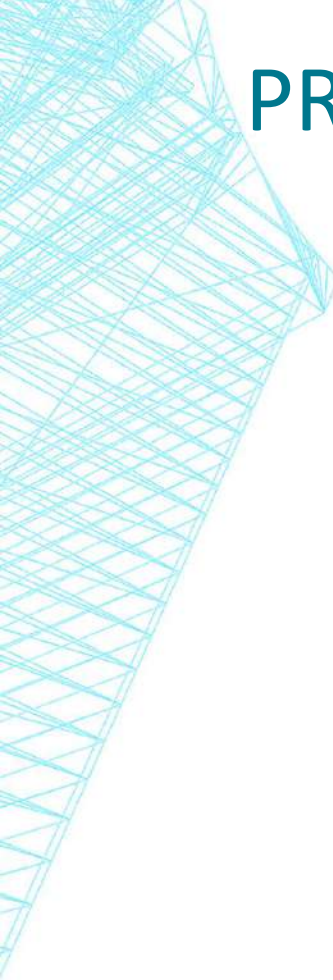


ALGORITMOS POLINOMIAIS E EXPONENCIAIS

- Algoritmos polinomiais:
 - Função de complexidade é $O(f(n))$, sendo $f(n)$ um polinômio de grau n .
- Algoritmos exponenciais:
 - Função de complexidade é $O(c^n)$, com $c > 1$.

ALGORITMOS POLINOMIAIS E EXPONENCIAIS

- A distinção entre estes dois tipos de algoritmos torna-se significativa quando o **tamanho do problema a ser resolvido cresce**.
- Esta é a razão porque algoritmos polinomiais **são muito mais úteis** na prática do que algoritmos exponenciais.
 - Geralmente, algoritmos exponenciais são simples variações de pesquisa exaustiva.



PROBLEMAS E ALGORITMOS

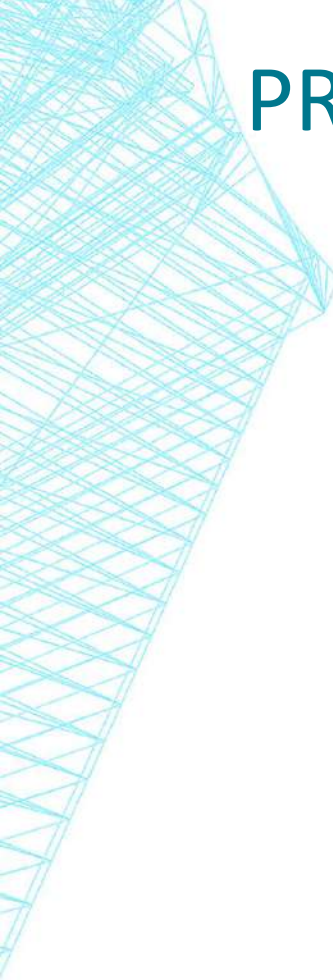
- Complexidade *intrínseca* de problemas computacionais.
- Melhor algoritmo possível $\times$ melhor algoritmo conhecido.
- Os algoritmos polinomiais são geralmente obtidos através de um entendimento mais profundo da estrutura do problema.

PROBLEMAS E ALGORITMOS

- Problemas fáceis x problemas difíceis:
 - Problemas tratáveis ou intratáveis.
 - Complexidade polinomial x complexidade exponencial.
- Teoria da complexidade de algoritmos:
 - **Objetivo:** identificar problemas difíceis.
 - Optar pelo uso de heurísticas por meio das técnicas de projeto de algoritmos.

PROBLEMAS E ALGORITMOS

- Um problema é considerado intratável se ele é tão difícil que não se conhece um algoritmo polinomial para resolvê-lo.
- Um problema é considerado tratável (bem resolvido) se existe um algoritmo polinomial para resolvê-lo.
- Aspecto importante no projeto de algoritmos.



PROBLEMAS FÁCEIS E DIFÍCEIS

- Diferença pode ser sutil:
 - Dado um conjunto de cidades, existe um caminho que saia de uma cidade A, passe por somente uma cidade intermediária C e chegue a uma cidade B?

PROBLEMAS FÁCEIS E DIFÍCEIS

- Diferença pode ser sutil:
 - Dado um conjunto de cidades, qual é o menor caminho que liga a cidade A à cidade B, passando por todas as demais cidades, sem repetição?

PROBLEMAS FÁCEIS E DIFÍCEIS

- A distinção entre algoritmos polinomiais eficientes e algoritmos exponenciais ineficientes possui **várias exceções**.
- Exemplo: um algoritmo com função de complexidade $f(n) = 2^n$ é mais rápido que um algoritmo $g(n) = n^5$ para valores de $n \leq 20$.
- Também existem algoritmos exponenciais que são muito úteis na prática.
 - Exemplo: o algoritmo Simplex para programação linear possui complexidade de tempo exponencial para o pior caso mas executa muito rápido na prática.
- Tais exemplos não ocorrem com frequência na prática, e muitos algoritmos exponenciais conhecidos não são muito úteis.

DECISÃO X OTIMIZAÇÃO

- Em um problema de **otimização** queremos determinar uma solução possível com o melhor valor.
- Em um problema de **decisão** queremos responder “sim” ou “não”.
- Para cada problema de otimização podemos encontrar um problema de decisão equivalente a ele.



CLASSES DE PROBLEMAS

- Problemas polinomiais (P)
- Problemas não-determinísticos polinomiais (NP)
- Problemas NP-Completo
- Problemas NP-Difíceis

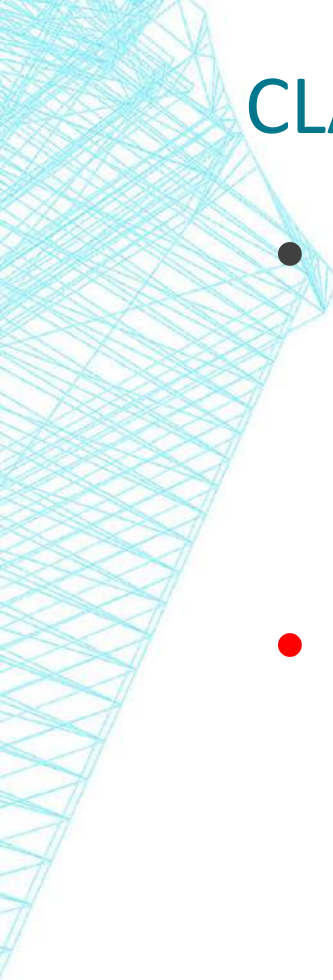
PROBLEMAS POLINOMIAIS – CLASSE P

- Abrange os problemas que podem ser resolvidos em tempo polinomial.
- Ex.:
 - Ordenação de dados.
 - Verificar se um grafo é completo.
 - Pesquisar um eleitor na base do TSE.

PROBLEMAS POLINOMIAIS – CLASSE *P*

Verificação de Ordenação está em NP

```
VOrdenacao(A,n)
inicio
    ordenado ← verdadeiro
    para i ← 1 até n-1 faça
        se A[i] > A[i+1] então
            ordenado ← falso
        fim se
    fim para
    se ordenado = falso então
        escreva "NAO"
    senão
        escreva "SIM"
    fim se
fim
```



CLASSE *NP* (*NONDETERMINISTIC POLYNOMIAL TIME*):

- Problemas para os quais não existe algoritmo polinomial conhecido para solução com *algoritmo determinista*.
- O que é um algoritmo determinista?

CLASSE *NP* (*NONDETERMINISTIC POLYNOMIAL TIME*):

- Problemas para os quais não existe algoritmo polinomial conhecido para solução com algoritmo determinista.
- Em termos simples: se você executar o algoritmo várias vezes com a mesma entrada, ele seguirá exatamente a mesma sequência de passos e produzirá sempre o mesmo resultado. Não há espaço para o acaso, escolhas aleatórias ou "palpites".

CLASSE *NP* (*NONDETERMINISTIC POLYNOMIAL TIME*):

- Um algoritmo não-determinista é capaz de **escolher uma dentre as várias alternativas** possíveis a cada passo.
- Utilizam:
 - a função `escolhe(C)`: escolhe um dos elementos de C de forma arbitrária.
 - SUCESSO: sinaliza uma computação **com sucesso**
 - INSUCESSO: sinaliza uma computação **sem sucesso**

CLASSE *NP* (*NONDETERMINISTIC POLYNOMIAL TIME*):

- Um algoritmo não-determinista é capaz de **escolher uma dentre as várias alternativas** possíveis a cada passo.
- Utilizam:
 - a função `escolhe(C)`: escolhe um dos elementos de `C` de forma arbitrária.
 - `SUCESSO`: sinaliza uma computação **com sucesso**
 - `INSUCESSO`: sinaliza uma computação **sem sucesso**
- Sempre que existir um conjunto de opções que levam a um término com sucesso, então, este conjunto é sempre escolhido
- A complexidade da função `escolhe` é $\Theta(1)$

EXEMPLO: PESQUISA

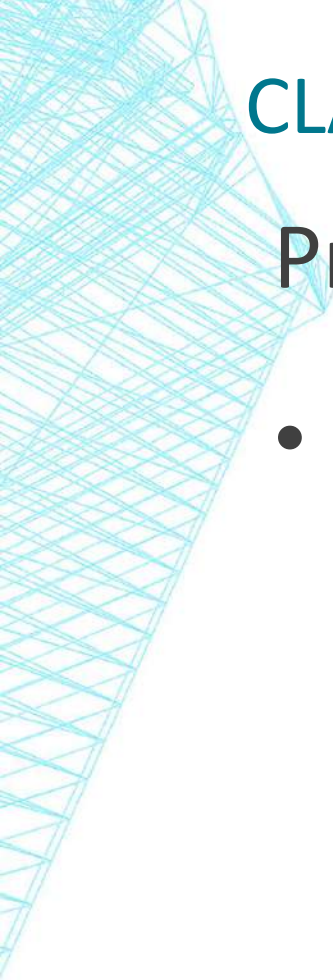
- Pesquisar o elemento x em um conjunto de elementos $A[1..n], n \geq 1$

```
j ← escolhe (A, x)
se A[j] = x então
    SUCESSO
senão
    INSUCESSO
fim se
```

- Complexidade: $\Theta(1)$
- Para um algoritmo determinista a complexidade é $\Theta(n)$

CLASSE *NP* (NONDETERMINISTIC POLYNOMIAL TIME):

- “Problemas para os quais não existe algoritmo polinomial conhecido para solução com algoritmo determinista.”
- No entanto, se for dada uma solução, ela pode ser verificável facilmente.
- Problema de decisão.



CLASSE ***NP*** (*NONDETERMINISTIC POLYNOMIAL TIME*):

Problema de ***decisão***.

- Ex: existe caminho entre A e B menor que K?

CLASSE *NP* (*NONDETERMINISTIC POLYNOMIAL TIME*):

Problema de *decisão*.

- Ex: existe caminho entre A e B menor que K?

existe um caminho que passe por todas as cidades sem repetição? (*ciclo Hamiltoniano*)

CLASSE *NP* (*NONDETERMINISTIC POLYNOMIAL TIME*):

Problema de *decisão*.

- Ex: existe caminho entre A e B menor que K?

 existe um caminho que passe por todas as
 cidades sem repetição? (*ciclo Hamiltoniano*)

 existe uma soma de valor K dos elementos de um
 conjunto S?

COMO MOSTRAR QUE UM DETERMINADO PROBLEMA ESTÁ EM NP?

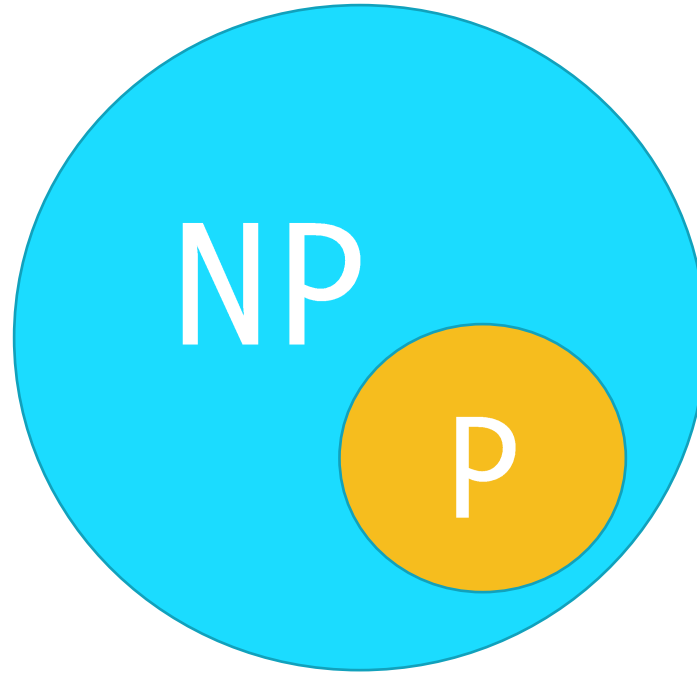
- Basta apresentar um algoritmo não-determinista que execute em tempo polinomial para **resolver o problema**.

ou

- Basta encontrar um algoritmo determinista polinomial para **verificar que uma dada solução é válida**.

CLASSES P E NP

- $P \subseteq NP$



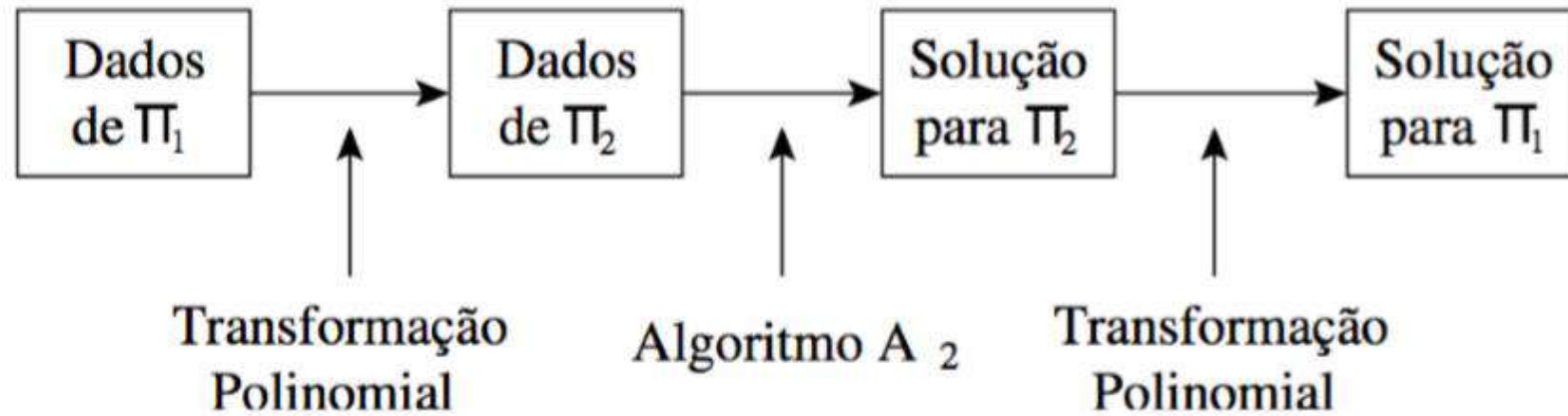
$P = NP?$

- Existem algoritmos polinomiais deterministas para os problemas em NP?
 - Caso sim, então $P=NP$.
 - Caso não, então $P \neq NP$.
 - e nenhum limite inferior não-polinomial foi provado para vários problemas NP.

TRANSFORMAÇÃO POLINOMIAL

- Considere dois problemas de decisão, Π_1 e Π_2
- Suponha que existe um algoritmo polinomial A_2 para resolver Π_2
- Se pudermos transformar polinomialmente Π_1 em Π_2 e a solução de Π_2 em solução de Π_1 , então A_2 pode ser usado para resolver Π_1

TRANSFORMAÇÃO POLINOMIAL



- Π_1 é polinomialmente transformável em Π_2

$$\Pi_1 \alpha \Pi_2$$

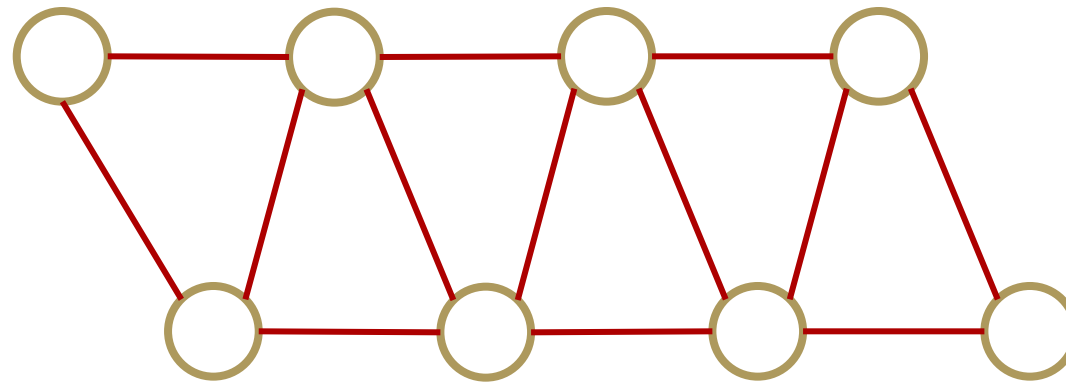
TRANSFORMAÇÃO POLINOMIAL

- Dois problemas Π_1 e Π_2 são considerados polinomialmente equivalentes se:

$$\Pi_1 \alpha \Pi_2 \text{ e } \Pi_2 \alpha \Pi_1$$

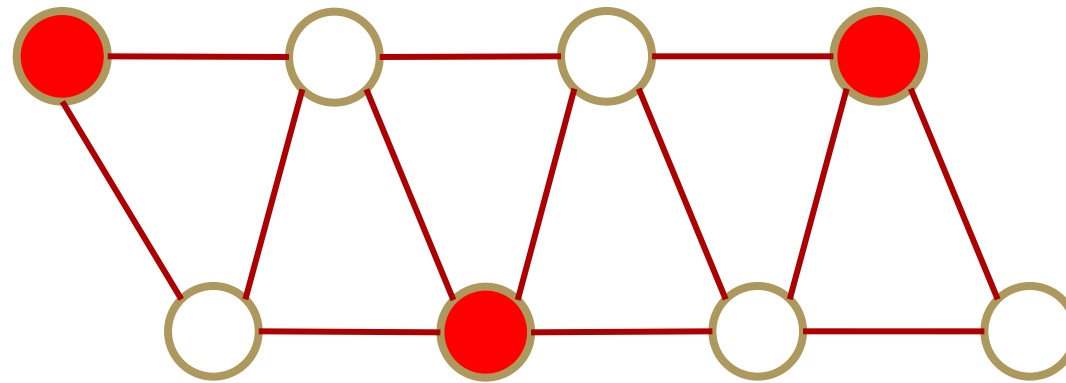
TRANSFORMAÇÃO POLINOMIAL

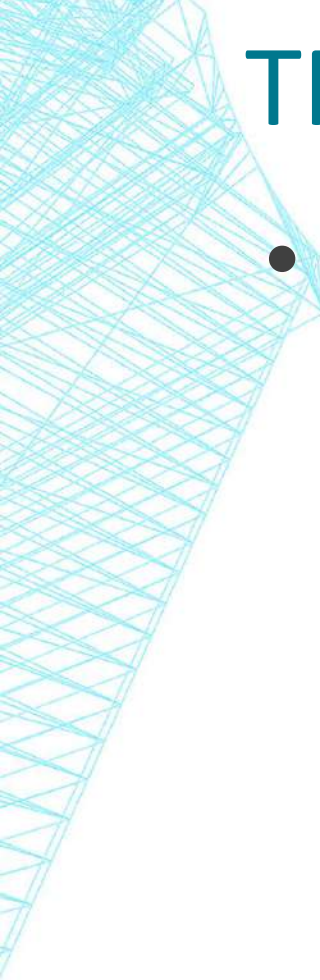
- Conjunto independente de vértices de um grafo:
 - Subconjunto de vértices V' do grafo no qual nenhum par de vértices de V' é adjacente



TRANSFORMAÇÃO POLINOMIAL

- Conjunto independente de vértices de um grafo:
 - Subconjunto de vértices V' do grafo no qual nenhum par de vértices de V' é adjacente



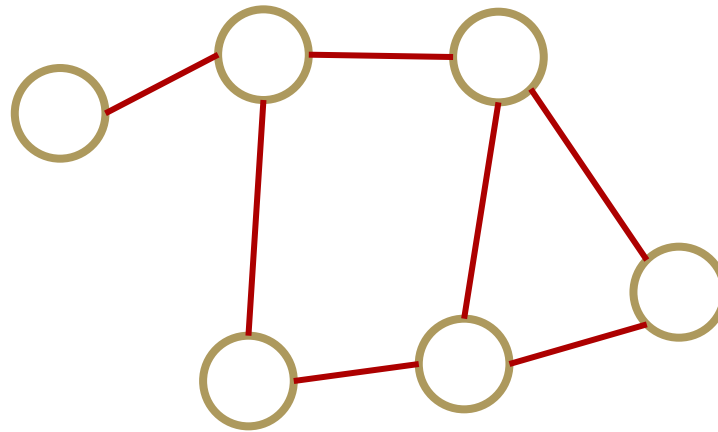


TRANSFORMAÇÃO POLINOMIAL

- Clique em um grafo
 - Um subconjunto C dos vértices de um grafo é uma clique do grafo se C é um subgrafo completo do grafo

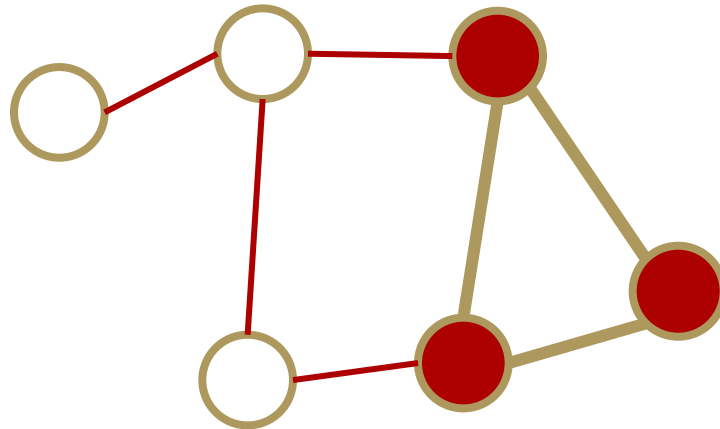
TRANSFORMAÇÃO POLINOMIAL

- Clique em um grafo
 - Um subconjunto C dos vértices de um grafo é uma clique do grafo se C é um subgrafo completo do grafo



TRANSFORMAÇÃO POLINOMIAL

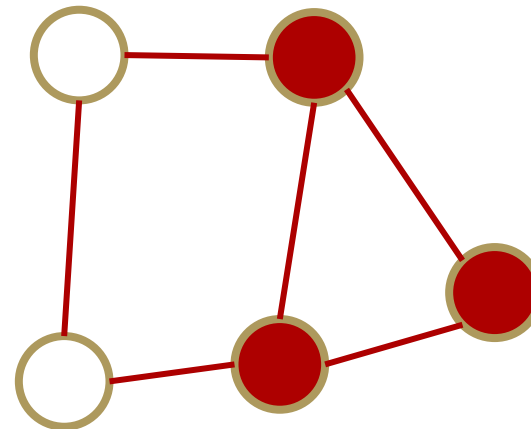
- Clique em um grafo
 - Um subconjunto C dos vértices de um grafo é uma clique do grafo se C é um subgrafo completo do grafo



TRANSFORMAÇÃO POLINOMIAL

- Considere Π_1 o problema da clique e Π_2 o problema do conjunto independente.

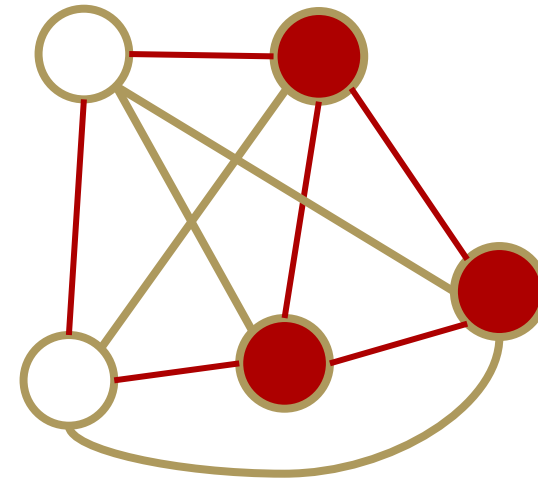
1) Uma instância I de clique consiste em um subgrafo com $k > 0$



TRANSFORMAÇÃO POLINOMIAL

- Considere Π_1 o problema da clique e Π_2 o problema do conjunto independente.

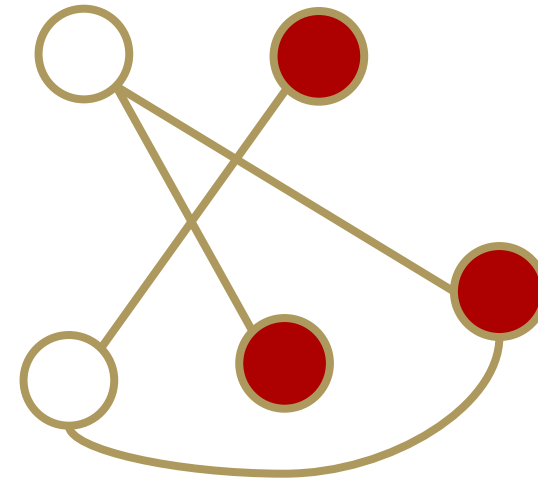
2) Obtendo o grafo complementar:



TRANSFORMAÇÃO POLINOMIAL

- Considere Π_1 o problema da clique e Π_2 o problema do conjunto independente.

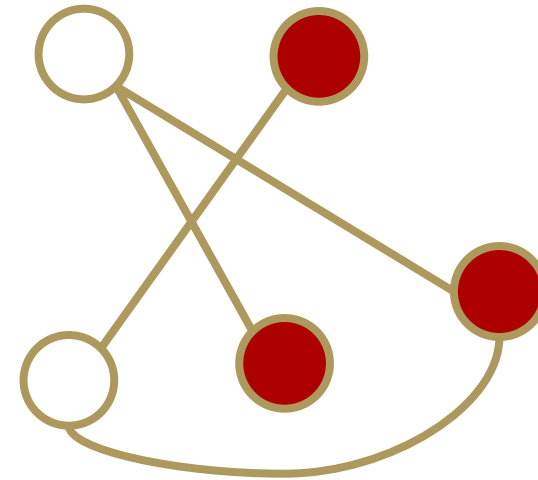
2) Obtendo o grafo complementar:



TRANSFORMAÇÃO POLINOMIAL

- Considere Π_1 o problema da clique e Π_2 o problema do conjunto independente.

3) Um clique em G é um conjunto independente em $\bar{G}$:



TRANSFORMAÇÃO POLINOMIAL

- Considere Π_1 o problema da clique e Π_2 o problema do conjunto independente.

3) Um clique em G é um conjunto independente em $\bar{G}$:

É possível obter o grafo complementar de G em tempo polinomial?

TRANSFORMAÇÃO POLINOMIAL

- Considere Π_1 o problema da clique e Π_2 o problema do conjunto independente.

3) Um clique em G é um conjunto independente em $\bar{G}$:

É possível obter o grafo complementar de G em tempo polinomial?

- SIM $\Rightarrow$ clique α conjunto independente.
- Um algoritmo que resolve clique, resolve o conjunto independente e vice-versa.

PROBLEMAS NP-COMPLETOS

- Um problema de decisão Π é dito NP-Completo se:
 1. $\Pi \in \text{NP}$
 2. Todo problema de decisão $\Pi' \in \text{NP-Completo}$ satisfaz $\Pi' \alpha \Pi$ ($\alpha = \text{'reduz-se polinomialmente a'}$)

PROBLEMAS NP-COMPLETOS

Para classificar um problema Π como **NP-Completo**, devemos satisfazer duas condições obrigatórias:

Passo 1: Provar que $\Pi \in NP$ (Pertencimento)

- **O que fazer:** Mostrar que o problema é "fácil de conferir".
- **Ação:** Dado um certificado (uma solução candidata), demonstre que existe um **algoritmo determinista** que verifica se essa solução é válida em **tempo polinomial**.
- *Exemplo:* Dado um caminho no Caixeiro Viajante, você consegue somar o peso das arestas rapidamente? Se sim, está em NP .

PROBLEMAS NP-COMPLETOS

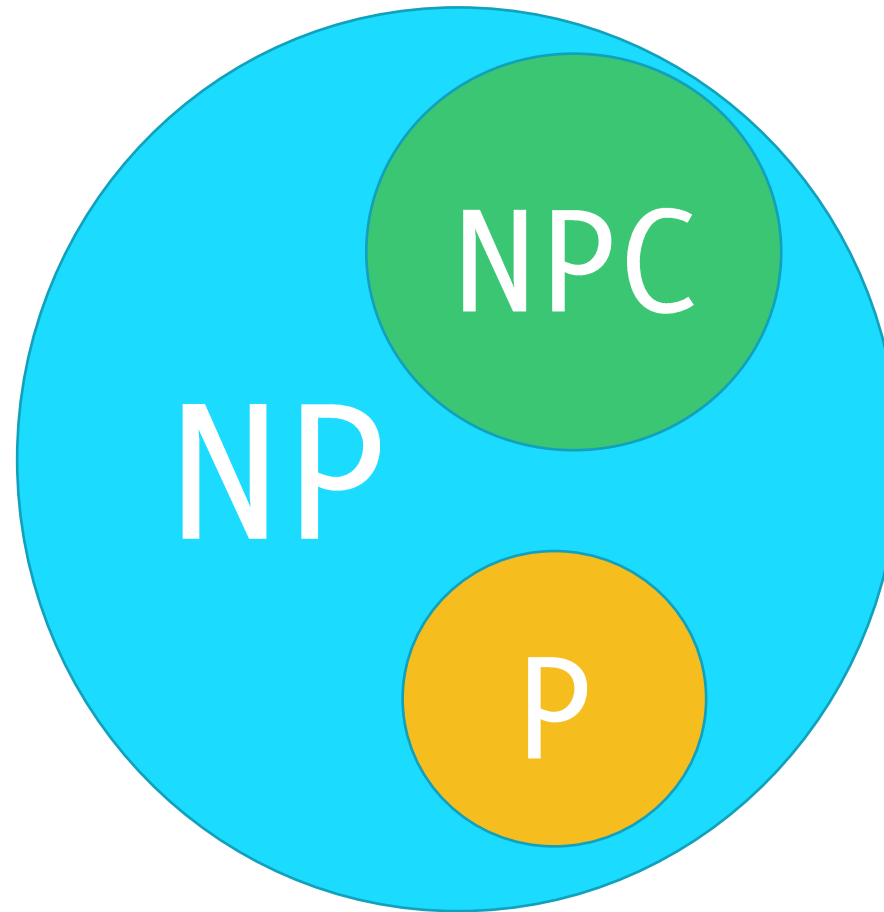
Passo 2: Provar que Π é NP-Difícil (Redutibilidade)

- **O que fazer:** Mostrar que Π é "tão difícil quanto" os problemas mais difíceis de NP
- **Ação:** Escolha um problema Π' que já sabemos ser NP-Completo (ex: SAT, 3-SAT, Clique) e mostre que:

$$\Pi' \propto \Pi$$

- *Crie uma função que transforma qualquer instância de Π' em uma instância de Π em tempo polinomial.*
- Prove que se a resposta para Π' é "SIM", a resposta para Π também será "SIM" (e vice-versa).
- **Conclusão:** Se o problema cumpre o **Passo 1** e o **Passo 2**, ele é oficialmente NP-Completo.

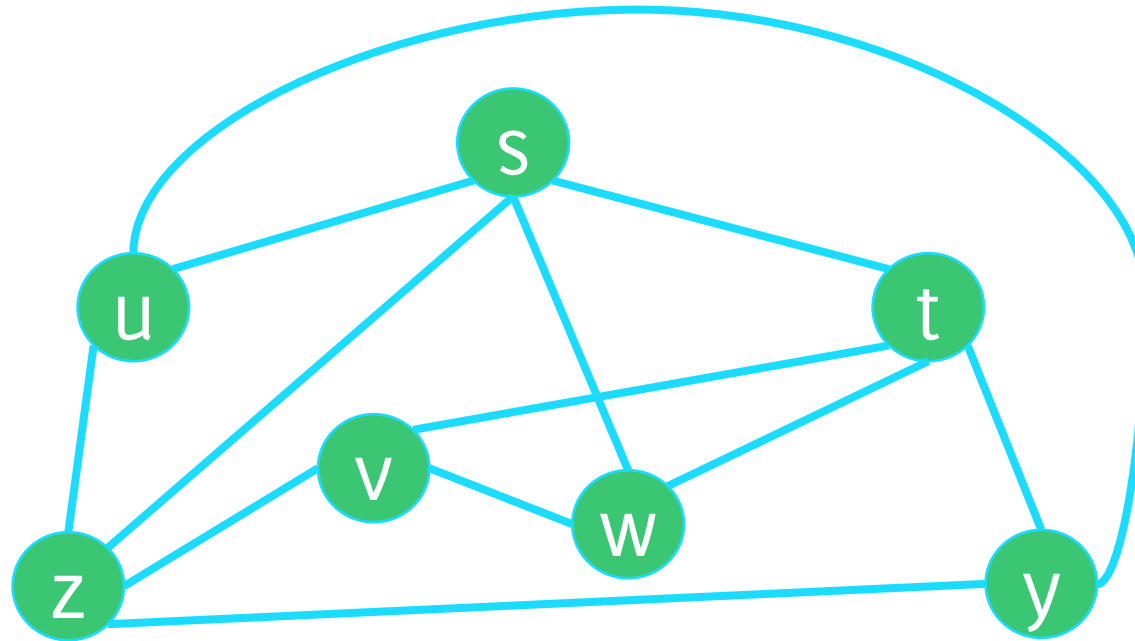
P, NP E NP-COMPLETO



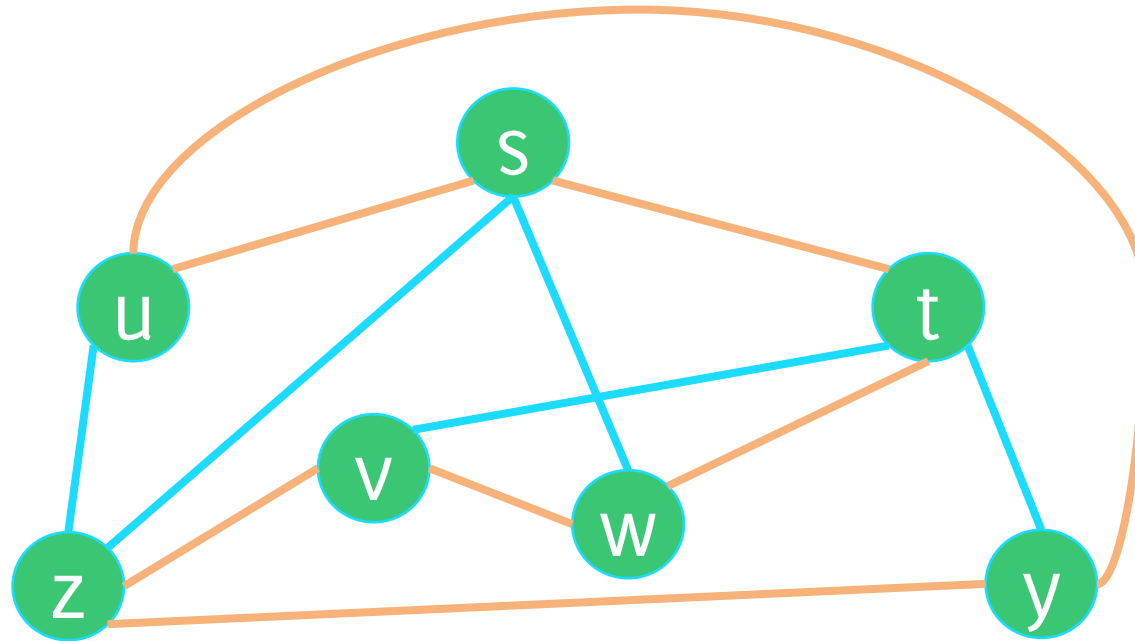
DECISÃO X OTIMIZAÇÃO

- Problemas de decisão x problema de otimização:
 - Existe um caminho fechado que passe por todas as cidades sem repetir? (decisão).
 - Determinar um caminho fechado que passe por todas as cidades sem repetir nenhuma com custo total mínimo (otimização).

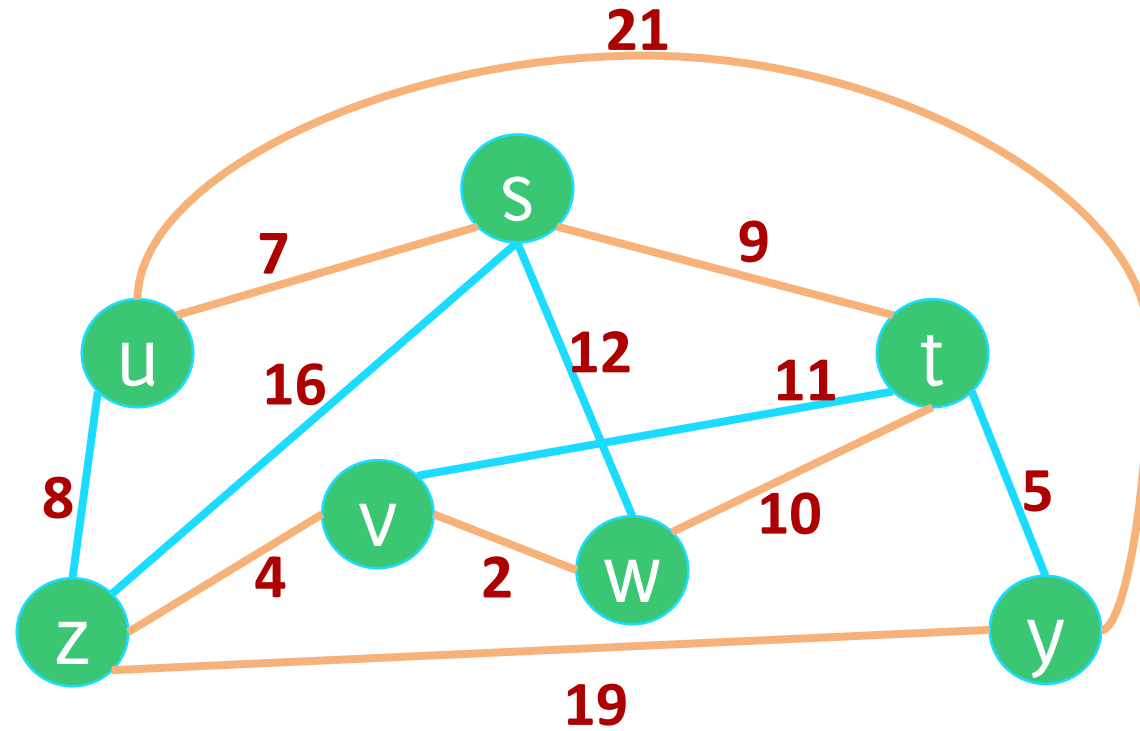
EXEMPLO – CICLO HAMILTONIANO



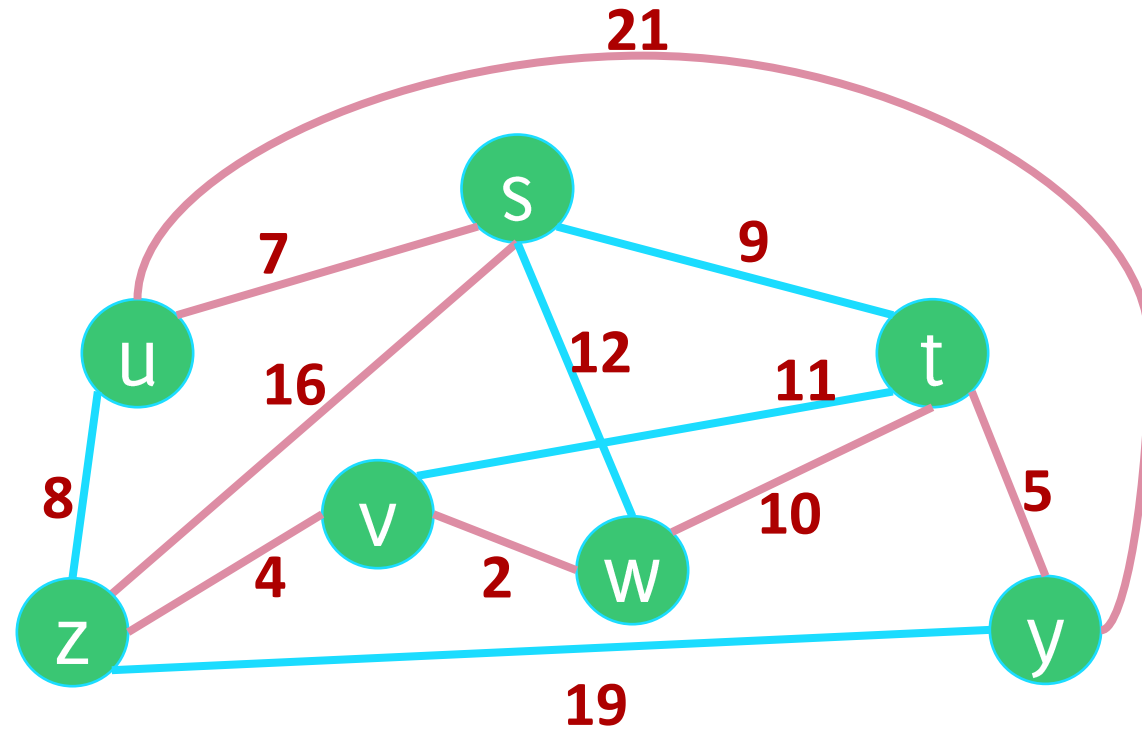
EXEMPLO – CICLO HAMILTONIANO



QUAL É O MENOR CICLO HAMILTONIANO?



QUAL É O MENOR CICLO HAMILTONIANO?



PROBLEMAS NP-DIFÍCEIS (NP-HARD)

- Caso um problema satisfaça a condição 2, mas não a condição 1, ele é dito NP-Difícil.
 1. $\Pi \notin \text{NP}$
 2. Todo problema de decisão $\Pi' \in \text{NP-Completo}$ satisfaz $\Pi' \leq \Pi$

PROBLEMAS NP-DIFÍCEIS (NP-HARD)

As implicações disso são:

- **Poder de Resolução:** Ele é "pelo menos tão difícil quanto" qualquer problema em NP . Se você encontrasse um algoritmo polinomial para resolver esse problema Π , você automaticamente resolveria todos os problemas da classe NP em tempo polinomial (provando que $P=NP$).
- **Dificuldade de Verificação:** A principal diferença de ele não estar em NP é que, dada uma solução candidata, **não existe (ou não conhecemos) um algoritmo polinomial que consiga verificar se essa solução está correta.**
- **Além dos Problemas de Decisão:** **Problemas NP-Difíceis não precisam ser problemas de "Sim ou Não".** Eles podem ser problemas de **otimização** (ex: "Qual o menor caminho?") ou até problemas **indecidíveis** (como o Problema da Parada, que é NP-Difícil mas impossível de resolver por um computador).

PROBLEMAS NP-DIFÍCEIS (NP-HARD)

